

Implementación de VCO usando RP2040

Dr. Pedro E. Colla (LU7DZ)

pedro.colla@gmail.com

Proyecto: PixiePico

4 de septiembre de 2026

Resumen

Esta nota técnica documenta el desarrollo de un algoritmo para implementar un VCO (DDS) basado en la utilización de un procesador ARM rp2040 tal como el disponible en placas rp2040 Zero o Raspberry Pico. El VCO utiliza un PIO entre los 8 disponibles en la arquitectura para generar una onda cuadrada a partir del control del clock de ejecución y los divisores disponibles. Se analiza el error obtenible en distintos escenarios y su aplicabilidad en el rango de frecuencias de HF.

1. Introducción

Este documento refleja el desarrollo realizado como parte del proyecto PixiePico para implementar un generador de señales que pueda funcionar en el rango de HF a partir de un procesador ARM rp2040. El trabajo se basa en lo desarrollado para el proyecto [ADXddsPIO](#) antecedentes de técnicas similares se pueden encontrar en el proyecto [pico – WSPR – tx](#). Se aprovecha la disponibilidad en el procesador ARM rp2040 de hasta 8 procesadores auxiliares RISC denominados PIO, cada uno de ellos con su propio espacio de ejecución y firmware modificable.

2. Planteo del problema

El oscilador se construye a partir de una máquina de estados PIO disponible en el procesador rp2040. Esa máquina se configura para utilizar el clock del procesador PLL_SYS (f_{SYS}) como fuente de reloj de la máquina de estados. El programa PIO ejecuta dos estados por cada período de salida: uno mantiene la señal alta y el siguiente la mantiene baja. Por lo tanto, si D es el divisor de reloj de la PIO,

$$f_{out} = \frac{f_{SYS}}{2D}. \quad (1)$$

El divisor utilizado para generar el clock del PIO a partir del clock principal se representa en formato Q16.8:

$$D = d_{int} + \frac{d_{frac}}{256} = \frac{N}{256}, \quad N = 256d_{int} + d_{frac}, \quad (2)$$

donde $0 \leq d_{frac} \leq 255$. Sustituyendo (2) en (1):

$$f_{out} = \frac{128f_{SYS}}{N}. \quad (3)$$

3. Modelo del PLL del RP2040

Para una referencia externa f_{XOSC} , el PLL genera

$$f_{\text{ref}} = \frac{f_{\text{XOSC}}}{\text{REFDIV}}, \quad (4)$$

$$f_{\text{VCO}} = f_{\text{ref}} \text{FBDIV}, \quad (5)$$

$$f_{\text{SYS}} = \frac{f_{\text{VCO}}}{\text{POSTDIV1} \text{POSTDIV2}}. \quad (6)$$

La documentación del RP2040 establece las siguientes restricciones del PLL [1]:

$$f_{\text{ref}} \geq 5 \text{ MHz}, \quad (7)$$

$$750 \text{ MHz} \leq f_{\text{VCO}} \leq 1.600 \text{ MHz}, \quad (8)$$

$$16 \leq \text{FBDIV} \leq 320, \quad (9)$$

$$1 \leq \text{POSTDIV1}, \text{POSTDIV2} \leq 7. \quad (10)$$

Se adopta además $\text{POSTDIV1} \geq \text{POSTDIV2}$, tal como recomienda la documentación.

Con $f_{\text{XOSC}} = 12 \text{ MHz}$, la primera restricción implica

$$\frac{12}{\text{REFDIV}} \geq 5 \implies \boxed{\text{REFDIV} \in \{1, 2\}}. \quad (11)$$

Por consiguiente se puede fijar REFDIV como parte de la solución mientras genere soluciones válidas.

El intervalo 125 MHz a 250 MHz para f_{SYS} es un requisito experimental del proyecto PixiePico derivado de experimentar con placas rp2040 Zero. El rango superior representa un "overclock" significativo de la placa pues la misma opera en forma nominal a $f_{\text{SYS}} = 125 \text{ MHz}$, sin embargo se comprueba empíricamente que opera bien en esas frecuencias. Algunas placas Raspberry Pico se pueden hacer operar en el intervalo mas amplio 125 MHz a 250 MHz pero esa diferencia no contribuye significativamente a la solución del problema.

4. Función objetivo

Combinando (3) y (6) se obtiene

$$\boxed{f_{\text{out}} = \frac{128 f_{\text{XOSC}} \text{FBDIV}}{\text{REFDIV} \text{POSTDIV1} \text{POSTDIV2} N}} \quad (12)$$

Para una frecuencia objetivo f_T , el problema consiste en encontrar la tupla entera

$$(\text{REFDIV}, \text{FBDIV}, \text{POSTDIV1}, \text{POSTDIV2}, N) \quad (13)$$

que minimiza

$$E = |f_{\text{out}} - f_T|, \quad (14)$$

sujeta a todas las restricciones anteriores y a

$$125 \text{ MHz} \leq f_{\text{SYS}} \leq 250 \text{ MHz}. \quad (15)$$

No es apropiado optimizar primero el PLL y luego la PIO de manera independiente. Una frecuencia f_{SYS} que no sea la más cercana a un valor elegido a priori puede producir un error final menor al combinarse con cierto valor entero N .

5. Algoritmo de búsqueda

El espacio de parámetros es discreto y pequeño. Por ello, una enumeración exhaustiva entrega el óptimo global dentro del modelo.

Para cada cuádrupla válida de parámetros PLL se calcula

$$N^* = \frac{128f_{\text{SYS}}}{f_T}. \quad (16)$$

Como el error es monótono a cada lado de N^* , basta evaluar $\lfloor N^* \rfloor$ y $\lceil N^* \rceil$. No es necesario recorrer los más de dieciséis millones de valores posibles del registro PIO.

Se utiliza aritmética entera, El uso prematuro de aritmética `float` puede cambiar el orden de candidatos con errores muy próximos.

El código Python que permite evaluar las soluciones para una dada frecuencia objetivo y el error que se puede optimizar es el siguiente:

Listing 1: Implementación de referencia del optimizador

```
from fractions import Fraction

XOSC = 12_000_000
SYS_MIN = 125_000_000
SYS_MAX = 250_000_000
VCO_MIN = 750_000_000
VCO_MAX = 1_600_000_000

def optimize(target_hz):
    best = None

    for refdiv in range(1, 64):
        ref = Fraction(XOSC, refdiv)
        if ref < 5_000_000:
            continue

        for fbdiv in range(16, 321):
            vco = ref * fbdiv
            if not (VCO_MIN <= vco <= VCO_MAX):
                continue
            if ref > vco / 16:
                continue

            for postdiv1 in range(1, 8):
                for postdiv2 in range(1, postdiv1 + 1):
                    sys = vco / (postdiv1 * postdiv2)
                    if not (SYS_MIN <= sys <= SYS_MAX):
                        continue

                    ideal_n = 128 * sys / target_hz
                    n0 = ideal_n.numerator // ideal_n.denominator

                    for n in {n0, n0 + 1}:
                        if not (256 <= n <= 0xFFFFF):
                            continue

                        actual = 128 * sys / n
                        signed_error = actual - target_hz

                        # Mayor VCO como desempate: menor jitter.
                        key = (abs(signed_error), -vco)
```

```

candidate = (key, reldiv, fbdiv,
             postdiv1, postdiv2,
             n, vco, sys,
             actual, signed_error)

if best is None or key < best[0]:
    best = candidate

return best

```

6. Resultados

Cuadro 1: Configuraciones óptimas encontradas para frecuencias testigo.

f_T (Hz)	REF	FB	PD1	PD2	N	f_{SYS} (Hz)	Error (Hz)
7.074.000	2	263	7	1	4079	225.428.571,429	+2,731762
14.074.000	2	211	3	2	1919	211.000.000,000	-3,126628
28.074.000	2	261	4	3	595	130.500.000,000	-50,420168

6.1. Caso $f_T = 7,074$ MHz

La mejor tupla es

$$(REFDIV, FB DIV, POSTDIV1, POSTDIV2, N) = (2, 263, 7, 1, 4079). \quad (17)$$

De ella se obtiene

$$f_{\text{ref}} = \frac{12 \text{ MHz}}{2} = 6 \text{ MHz}, \quad (18)$$

$$f_{\text{VCO}} = 6 \text{ MHz} \times 263 = 1.578 \text{ MHz}, \quad (19)$$

$$f_{\text{SYS}} = \frac{1.578 \text{ MHz}}{7} = 225,428.571.428.571 \text{ MHz}. \quad (20)$$

El divisor PIO es

$$D = \frac{4079}{256} = 15 + \frac{239}{256} = 15,93359375. \quad (21)$$

Finalmente,

$$f_{\text{out}} = 7.074.002,731.762 \text{ Hz}, \quad (22)$$

$$\Delta f = +2,731.762 \text{ Hz}, \quad (23)$$

$$\varepsilon = +0,386169 \text{ ppm}. \quad (24)$$

Como comprobación, 233 MHz también resulta sintetizable cuando se permite **REFDIV=2**: (2, 233, 3, 2) produce un VCO de 1.398 MHz y $f_{\text{SYS}} = 233$ MHz. Con $N = 4216$, su error es +3,795.066 Hz, ligeramente mayor que el óptimo global. El error obtenible es compatible con la utilización del generador en FT8 y CW.

6.2. Caso $f_T = 14,074$ MHz

La mejor tupla es

$$(REFDIV, FB DIV, POSTDIV1, POSTDIV2, N) = (2, 211, 3, 2, 1919). \quad (25)$$

Los relojes y el divisor PIO son

$$f_{\text{ref}} = 6 \text{ MHz}, \quad (26)$$

$$f_{\text{VCO}} = 1.266 \text{ MHz}, \quad (27)$$

$$f_{\text{SYS}} = \frac{1.266 \text{ MHz}}{6} = 211 \text{ MHz}, \quad (28)$$

$$D = \frac{1919}{256} = 7 + \frac{127}{256} = 7,49609375. \quad (29)$$

La salida resultante es

$$f_{\text{out}} = \frac{128(211000000)}{1919} = 14.073.996,873.372 \text{ Hz}, \quad (30)$$

$$\Delta f = \boxed{-3,126.628 \text{ Hz}}, \quad (31)$$

$$\varepsilon = \boxed{-0,222156 \text{ ppm}}. \quad (32)$$

El error obtenible es compatible con la utilización del generador en FT8 y CW.

6.3. Caso $f_T = 28,074 \text{ MHz}$

Una de las tuplas óptimas es

$$\boxed{(REFDIV, FB DIV, POSTDIV1, POSTDIV2, N) = (2, 261, 4, 3, 595)}. \quad (33)$$

Los valores asociados son

$$f_{\text{ref}} = 6 \text{ MHz}, \quad (34)$$

$$f_{\text{VCO}} = 1.566 \text{ MHz}, \quad (35)$$

$$f_{\text{SYS}} = \frac{1.566 \text{ MHz}}{12} = 130,5 \text{ MHz}, \quad (36)$$

$$D = \frac{595}{256} = 2 + \frac{83}{256} = 2,32421875. \quad (37)$$

La frecuencia obtenida es

$$f_{\text{out}} = \frac{128(130500000)}{595} = 28.073.949,579.832 \text{ Hz}, \quad (38)$$

$$\Delta f = \boxed{-50,420.168 \text{ Hz}}, \quad (39)$$

$$\varepsilon = \boxed{-1,795974 \text{ ppm}}. \quad (40)$$

Existen varias tuplas con el mismo error para este objetivo. Por ejemplo, el mismo VCO de 1.566 MHz con $(POSTDIV1, POSTDIV2, N) = (6, 2, 595)$, $(5, 2, 714)$ o $(7, 1, 1020)$ conserva la misma razón final. La selección entre ellas puede realizarse según frecuencia del sistema, consumo y compatibilidad con el resto del firmware. El error obtenible es compatible con la utilización del generador en CW pero no en FT8.

7. Limitaciones

1. La búsqueda obtiene el óptimo global *estático* dentro de las restricciones establecidas, el error no es cero pero se evalúa dentro de rangos que los hace compatibles con los principales usos anticipados (CW y modos digitales de baja señal).
2. El valor nominal calculado no incluye tolerancia, deriva térmica ni envejecimiento del cristal de 12 MHz. En la práctica, varios ppm del cristal pueden superar el error matemático residual del algoritmo.

3. Una frecuencia racional como 1.578 MHz/7 es físicamente válida. Sin embargo, las API del SDK que almacenan frecuencias en hertz enteros pueden reportarla redondeada. La configuración del PLL debe realizarse a partir de sus parámetros exactos, no reconstruirse desde el entero redondeado.
4. Cambiar PLL_SYS durante la ejecución afecta CPU, PIO y los periféricos derivados de ese reloj. Deben revisarse temporizadores, baud rates, flash y cualquier lógica que suponga una frecuencia fija. El PLL USB es independiente, aunque el firmware USB puede verse afectado indirectamente si el cambio no se secuencia correctamente.
5. Para errores menores que el óptimo estático la solución es alternar entre dos divisores PIO adyacentes en una proporción calculada lo que reduce el error promedio, pero introduce modulación de frecuencia (Jitter) y componentes laterales por lo que se analizará por separado su implementación utilizando un método de acumulación de Bresenham para poder intercalar dos tramas de clocks operando en DMA diferentes.

8. Implementación

El firmware debe conservar los cinco parámetros de la solución óptima, no solo f_{SYS} y D . La estructura de resultado debe incluir como mínimo `refdiv`, `fbdiv`, `postdiv1`, `postdiv2`, `pio_div_int`, `pio_div_frac`, frecuencia calculada y error. Antes de aplicar el resultado, debe verificarse nuevamente cada restricción y configurar el PLL mediante sus parámetros exactos. El diseño es adecuado para modos digitales de baja señal en bandas de HF hasta 14 MHz y hasta 28 MHz para CW.

Referencias

- [1] Raspberry Pi Ltd., *RP2040 Datasheet: A microcontroller by Raspberry Pi*, sección 2.18, “PLL”, versión de febrero de 2025. Disponible en: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>.
- [2] Raspberry Pi Ltd., *Pico SDK—hardware_pll*, implementación de `pll_init`. Disponible en: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_pll/pll.c.
- [3] Transceiver digital modos de señales débiles https://github.com/lu7did/ADX_ddsPIO.
- [4] Transmisor digital usando VCO basado en rp2040 <https://github.com/RPiks/pico-WSPR-tx>.